

Types construits - Dictionnaires

Term NSI – Septembre 2026

Fiche d'exercices sur les tableaux et dictionnaires, algorithmes de somme et moyenne, compteur, fusion, base de donnée en JSON.

1 Dictionnaires — Exercices niveau Terminale NSI

1.1 Exercice 1 — Carnet de notes (dictionnaires imbriqués)

On modélise les notes d'une classe avec le dictionnaire suivant :

```
1 notes = {  
2     'Alice': {'maths': 15, 'nsi': 18, 'philosophie': 12},  
3     'Bilal': {'maths': 8, 'nsi': 14, 'philosophie': 16},  
4     'Chloé': {'maths': 12, 'nsi': 9, 'philosophie': 13}  
5 }
```

1. Écrire une fonction `moyenne(eleve, notes)` qui renvoie la moyenne des notes d'un élève donné.

On pourra utiliser la fonction `sum` qui renvoie la somme de la liste placée en argument. Par exemple, `sum(L)` fait la somme des éléments de `L`.

```
1 def moyenne(eleve, notes):  
2     valeurs = notes[eleve].values()  
3     return ...
```

2. Écrire une fonction `moyenne_generale(notes)` qui renvoie un dictionnaire associant à chaque élève sa moyenne générale, sous la forme d'une compréhension de dictionnaire (une seule ligne):

```
1 def moyenne_generale(notes):  
2     return {...}  
3  
4 > moyenne_generale(notes)  
5 {'Alice': 15.0, 'Bilal': 12.67, 'Chloé': 11.33}
```

3. Écrire une fonction `moyenne_matiere(notes, matiere)` qui renvoie la moyenne de la classe dans une matière donnée.

```
1 def moyenne_matiere(notes, matiere):  
2     valeurs = [notes[eleve][matiere] for ... in ... ]  
3     return ...
```

4. Que se passe-t-il si un élève n'a pas de note dans une matière (clé absente) ? Proposer une version robuste de `moyenne_matiere` qui ignore les élèves sans note dans cette matière, sans provoquer d'erreur (on pourra utiliser un test `if matiere in ...` avant d'accéder à la note).

```

1 def moyenne_matiere(notes, matiere):
2     valeurs = [notes[eleve][matiere] for ... in ... if matiere in ... ]
3     if not valeurs:
4         return ...
5     return ...

```

1.2 Exercice 2 – Compteur de fréquences

On souhaite analyser un texte pour en compter les mots.

```

1 texte = "le chat mange la souris le chien mange le chat"

```

1. Écrire une fonction `compte_mots(texte)` qui renvoie un dictionnaire associant à chaque mot le nombre de fois qu'il apparaît dans `texte`. (on utilisera `texte.split()`).

```

1 def compte_mots(texte):
2     compteur = {}
3     for mot in texte.split():
4         if mot in ... :
5             compteur[mot] = ...
6         else:
7             compteur[mot] = ...
8     return compteur

```

2. Écrire une fonction `mot_le_plus_frequent(compteur)` qui parcourt le dictionnaire pour trouver la clé de plus grande valeur.

```

1 def mot_le_plus_frequent(compteur):
2     meilleur_mot = None
3     meilleur_nb = -1
4     for mot, nb in ...:
5         if nb > ...:
6             meilleur_mot = ...
7             meilleur_nb = ..
8     return ...

```

3. Écrire une fonction `mots_frequents(compteur, seuil)` qui renvoie la liste des mots apparaissant au moins `seuil` fois, triée par ordre alphabétique.
4. **Complexité** : si le texte contient n mots, quelle est la complexité de `compte_mots` ? Justifier en s'appuyant sur la structure de table de hachage du dictionnaire Python.

1.3 Exercice 3 – Fusion et transformation de dictionnaires

```

1 stock_magasin_A = {'pommes': 30, 'poires': 12, 'bananes': 25}
2 stock_magasin_B = {'poires': 8, 'bananes': 10, 'kiwis': 15}

```

1. Écrire une fonction `fusion_stocks(a, b)` qui renvoie un nouveau dictionnaire contenant, pour chaque fruit présent dans `a` ou `b`, la somme des quantités des deux magasins (sans modifier `a` ni

b). On utilisera un test `if fruit in ...` pour savoir si le fruit a déjà une entrée dans le dictionnaire résultat.

```
1 def fusion_stocks(a, b):
2     resultat = dict(a) # copie de a pour ne pas le modifier
3     for fruit, quantite in b.items():
4         if fruit ...
5             resultat[fruit] += ...
6         else:
7             resultat[fruit] = ...
8     return ...
```

2. On veut connaître les fruits disponibles **uniquement** dans le magasin A et pas dans B. Écrire une fonction `uniquement_A(a, b)` qui renvoie la liste de ces fruits, à l'aide d'une boucle `for` et de l'opérateur `in`.
3. Écrire une fonction `quantite_totale(a, b)` qui renvoie la quantité totale de fruits présente dans les deux magasins réunis (tous fruits confondus).
4. Écrire une fonction `appliquer_remise(stock, prix, taux)` qui renvoie un dictionnaire associant à chaque fruit du stock son prix après une remise de `taux %` (on suppose l'existence d'un dictionnaire `prix` donnant le prix unitaire de chaque fruit). Utiliser une compréhension de dictionnaire.

1.4 Exercice 4 — Base de données imbriquée façon JSON

On modélise un petit catalogue de livres d'une bibliothèque :

```
1 bibliotheque = {
2     '978-2': {'titre': 'Le Petit Prince', 'auteur': 'Saint-Exupéry', 'annee':
3     1943, 'emprunte': False},
4     '978-3': {'titre': '1984', 'auteur': 'Orwell', 'annee': 1949, 'emprunte':
5     True},
6     '978-4': {'titre': 'Fondation', 'auteur': 'Asimov', 'annee': 1951,
7     'emprunte': False}
8 }
```

1. Écrire une fonction `livres_disponibles(bibliotheque)` qui renvoie la liste des titres des livres non empruntés.
2. Écrire une fonction `emprunter(bibliotheque, isbn)` qui marque un livre comme emprunté (`'emprunte': True`) si l'ISBN existe et que le livre est disponible ; la fonction renvoie `True` si l'emprunt a réussi, `False` sinon.

```
1 def emprunter(bibliotheque, isbn):
2     if isbn in bibliotheque and not bibliotheque[isbn]['emprunte']:
3         bibliotheque[isbn]['emprunte'] = True
4     return True
5     return False
```

3. Écrire une fonction `livres_par_auteur(bibliotheque)` qui renvoie un dictionnaire associant à chaque auteur la liste des titres qu'il a écrits (utile si un auteur a plusieurs livres dans la base). On testera si l'auteur est déjà une clé du dictionnaire résultat avec `if auteur in ...`.

```
1 def livres_par_auteur(bibliotheque):
2     resultat = {}
3     for info in bibliotheque.values():
4         auteur = info['auteur']
5         if auteur in resultat:
6             resultat[auteur].append(info['titre'])
7         else:
8             resultat[auteur] = [info['titre']]
9     return resultat
```

4. Écrire une fonction `livres_avant(bibliotheque, annee)` qui renvoie la liste des titres publiés strictement avant une année donnée.

```
1 def livres_avant(bibliotheque, annee):
2     livres = [info['titre'] for info in bibliotheque.values() if info['annee'] <
3     annee]
4     livres.sort()
5     return livres
6
7 > livres_avant(bibliotheque, 1984)
['1984', 'Fondation', 'Le Petit Prince']
```

5. **Ouverture** : ce genre de structure (dictionnaire de dictionnaires) correspond exactement au format d'échange **JSON**. Expliquer en une ou deux phrases pourquoi les dictionnaires Python se prêtent naturellement à la manipulation de formats JSON.

1.5 Exercice 5 — Vrai/Faux : dictionnaires et complexité

Pour chaque affirmation, dire si elle est vraie ou fausse et **justifier brièvement** :

1. Rechercher si une clé appartient à un dictionnaire (`cle in dico`) a une complexité en moyenne constante, $O(1)$.
 2. Les clés d'un dictionnaire Python peuvent être des listes.
 3. Depuis Python 3.7, l'ordre d'insertion des couples clé/valeur est conservé lors du parcours d'un dictionnaire.
 4. Deux dictionnaires ayant les mêmes couples clé/valeur mais insérés dans un ordre différent sont considérés comme égaux par l'opérateur `==`.
 5. La méthode `dico.pop(cle)` lève une exception si `cle` n'existe pas et qu'aucune valeur par défaut n'est fournie.
 6. Un dictionnaire est implémenté en Python à l'aide d'une **table de hachage**, ce qui explique pourquoi les clés doivent être *hachables* (immuables).
-